Oracle C2M Performance Tuning & Resource Optimization

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

The problem . . . 4

The approach . . . 4

What's inside . . . 4

Why it's safe . . . 4

The bottom line . . . 4

Solution Overview 5

The C2M performance model — four layers, top to bottom . . . 5

The measure-first method . . . 6

What's in this solution . . . 7

Who this is for . . . 7

Outcome . . . 7

Requirements Guide 7

1. Metrics, monitoring & a repeatable baseline . . . 7

2. Environment & administrative access (WebLogic, OUAF, database) . . . 8

3. A representative, repeatable workload profile . . . 8

4. Certified, matched stack versions across the tuning path . . . 9

5. Change control, backups & a non-production pilot . . . 9

6. Collaborating teams (DBA, infrastructure, app-dev) . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

Tuning Method 12

Step 0 — Prepare (Assess) . . . 12

Step 1 — Measure & baseline . . . 12

Step 2 — Isolate the bottleneck . . . 13

Step 3 — Tune the constraint (one layer, one change) . . . 13

Step 4 — Re-measure & validate . . . 15

Step 5 — Expand (promote & repeat) . . . 16

Testing Guide — Measured Gates 16

Gate targets are relative, not absolute . . . 16

Layer gates . . . 16
No-regression gate (mandatory, every pass) . . . 18
Production go/no-go gate . . . 18
Test execution checklist . . . 18
Troubleshooting Guide 18
Cross-cutting . . . 18
JVM / GC . . . 19
WebLogic threads / Work Managers . . . 19
JDBC connection pool . . . 19
Batch throughput . . . 20
SQL / Database . . . 20
Escalation . . . 21
Best Practices 21
Measure first, always . . . 21
One change at a time . . . 21
Respect layer coupling . . . 21
Prefer self-tuning and shaping over fixed counts . . . 22
Production discipline . . . 22
Version and licensing hygiene . . . 22
Keep the evidence . . . 22
Deliverables Summary 22
Documentation . . . 22
Automation scripts (WLST/Jython, shell, SQL) . . . 23
How they map to the method . . . 24
Outcome . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

The problem

As load grows, Oracle Utilities **Customer to Meter (C2M)** slows down — but the cause is rarely obvious. Performance problems surface across a stack of dependent layers: the **JVM and its garbage collection**, **WebLogic 14c thread pools and JDBC connection pools**, **OUAF batch throughput**, and the **Oracle Database and its SQL**. Tuning the wrong layer wastes scarce outage windows and can make throughput worse, because a fix in one layer often just pushes the bottleneck into the next.

The approach

This solution delivers a **measure-first tuning method** for C2M and its WebLogic 14c middleware: instrument the whole stack, find the *real* bottleneck from evidence, apply a single reversible change to the one constraining layer, then re-measure to prove the gain against a baseline — always on a **non-production pilot first**, under change control. It follows the standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model with explicit **measured pass/fail gates** and a documented rollback.

What's inside

- A **Solution Overview** of the four-layer C2M performance model and the measure-first method.
- A **Requirements Guide** covering the monitoring, access, workload, versioning, and change-control prerequisites — each with how to *verify*.
- A **Tuning Method**: baseline → isolate the bottleneck → tune JVM/GC, WebLogic thread & connection pools, batch thread pools, and SQL/database → re-measure and validate → expand.
- **Five automation scripts** (WLST/Jython, shell, SQL) — read-only discovery, read-only validation, a read-only evidence report, dry-run-by-default remediation, and a scope/rollback helper.
- **Testing and Troubleshooting** guides with measured gates, plus **Best Practices** for repeatable, evidence-based tuning.

Why it's safe

Every recommendation is grounded in Oracle's official WebLogic 14c and OUAF documentation; version-specific details are **cited or flagged "verify for your version,"** and My-Oracle-Support-gated notes are pointed to, not reproduced. The scripts that read live systems are **read-only**; remediation **defaults to dry-run**; and no change reaches production until it has passed a measured gate on a pilot with a rollback ready.

The bottom line

A slow C2M environment turned into a **measured, optimized** one — JVM, pools, batch, and SQL tuned to the real bottleneck, proven on a pilot before you scale. Self-guided and tested, at a fraction of the cost of a consulting engagement.

Solution Overview

Oracle Utilities **Customer to Meter (C2M)** runs on the **Oracle Utilities Application Framework (OUAF)** and is deployed on **Oracle WebLogic Server 14c** over an **Oracle Database**. When C2M "feels slow," the cause is almost never a single knob — it is a bottleneck somewhere in a stack of dependent layers, and turning the wrong knob wastes an outage window and can make throughput worse.

This solution gives you a **measure-first tuning method**: instrument the stack, find the *real* bottleneck with evidence, tune the one layer that is constraining you, then re-measure to prove the gain — and do it on a non-production pilot before you touch production.

> ***Version boundary.*** *Written for **C2M** on **OUAF** with **WebLogic Server 14c** (14.1.1) and an **Oracle Database**. Exact MBean paths, OUAF utility names, and default values vary by release — everything below is **cited or flagged "verify for your version."** Certain notes/patches are gated on **My Oracle Support (MOS)**; those are pointed to, not reproduced.*

The C2M performance model — four layers, top to bottom

Performance flows down a chain. A request (online transaction, web service, or a batch work unit) enters WebLogic, is handled by a JVM, holds a database connection from a pool, and executes SQL. Every layer can be the constraint. Tune **in dependency order** and only after you have measured which layer is actually saturated.

#	Layer	What it governs	Primary tuning artifacts
1	**JVM / GC**	The Java runtime every WebLogic node and every batch thread pool worker runs inside. Heap sizing and garbage-collection behavior set the ceiling for everything above it.	Heap (`-Xms`/`-Xmx`), GC collector + pause target (G1: `-XX:MaxGCPauseMillis`), metaspace, GC logging
2	**WebLogic pools**	How WebLogic dispatches work and lends out database connections: the **self-tuning thread pool / Work Managers**, and each **JDBC data source connection pool**.	Work Manager `min-threads-constraint` / `max-threads-constraint`; data source `Initial Capacity`, `Maximum Capacity`, `Test Connections on Reserve`, statement cache

#	Layer	What it governs	Primary tuning artifacts
3	**Batch throughput**	OUAF batch: how many **thread pool workers** run, how many **threads** a batch runs across, and how work is distributed. This is where nightly billing/meter volume lives.	Thread pool worker count and pool sizes (`threadpoolworker.sh -p`), per-batch thread count (`submitbatchREST.sh -c`), commit/`chunk` interval
4	**SQL / Database**	The queries C2M and its batches issue, their plans, indexes, and the database's own resource limits (sessions, CPU, I/O, undo).	AWR/ASH findings, execution plans, index/stat freshness, `processes`/`sessions`, connection-pool sizing back-pressure

The layers interact. Undersized DB connection pools (layer 2) masquerade as slow SQL (layer 4); GC pauses (layer 1) look like stuck WebLogic threads (layer 2); too many batch threads (layer 3) exhaust the connection pool (layer 2) and overload the database (layer 4). **This is exactly why the method is measure-first** — you diagnose before you tune.

The measure-first method

Guessing is the enemy. The method is a loop, run once per bottleneck, always on a **non-production pilot first** with change control and a rollback for production:

1. **Measure & baseline.** Capture a repeatable baseline of the whole stack under a representative

workload — GC behavior, WebLogic thread-pool and connection-pool runtime metrics, batch elapsed times/throughput, and a database AWR/ASH snapshot. Record the numbers; you will be judged against them. `OC2MPERF-Discovery` collects the read-only baseline for you.

1. **Isolate the bottleneck.** Read the baseline to find the **one** saturated layer: GC time as a

fraction of wall-clock, WebLogic `PendingUserRequestCount` / stuck/hogging threads, connection-pool `WaitingForConnectionCurrentCount`, batch thread saturation, or the top DB wait event. Do **not** tune more than one layer at a time.

1. **Tune the constraint.** Apply a **single, named, reversible** change to that layer only — a heap

or GC setting, a Work Manager or data-source capacity, a batch thread count, or a SQL/index fix.

1. **Re-measure & validate.** Re-run the *same* workload and compare against the baseline. Accept the

change only if it **moves the target metric past a defined gate** and does not regress another layer. Otherwise roll back.

1. **Expand.** Promote the validated change through the environments (Dev → Test → Prod) under change

control, then repeat the loop for the *next* bottleneck.

This maps onto the standard Oracle-category deployment model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit measured pass/fail gates and a documented rollback.

What's in this solution

• **Requirements Guide** — what you need before you tune (monitoring/metrics access, environment and

admin access, a representative workload profile, backups/change control), each with how to *verify*.

• **Tuning Method** — the ordered, hands-on procedure: baseline, isolate, tune JVM/GC → WebLogic

thread & connection pools → batch thread pools → SQL/DB, then re-measure and validate.

• **5 automation scripts (WLST/Jython, shell, SQL)** — read-only discovery, read-only per-node

validation, a read-only health/coverage report, a **dry-run-by-default** remediation helper, and a scope/target helper — authentic to C2M and WebLogic administration.

• **Testing & Troubleshooting Guides** with measured pass/fail gates, plus **Best Practices** for

repeatable, evidence-based tuning.

Who this is for

C2M / WebLogic administrators, OUAF technical operations staff, and the DBA and infrastructure teams who own the environment. It assumes you can reach the **WebLogic Administration Console / WLST**, the **OUAF product user shell**, and the **C2M database schema** — see the Requirements Guide.

Outcome

A slow C2M environment turned into a **measured, optimized** one: JVM/GC, WebLogic pools, batch, and SQL each tuned to the layer that was actually constraining you, every change proven against a baseline on a pilot before it reaches production.

Requirements Guide

These are the prerequisites specific to **performance tuning** C2M and its WebLogic 14c middleware. They are **in addition to** the shared Oracle-category prerequisites (licensed C2M/OUAF + certified WebLogic 14c and Oracle Database, defined Dev/Test/Prod environments, WLST/shell/SQL admin access, and change control/backup/non-production pilot) documented in the global Oracle requirements — verify those first. Each requirement below is stated as **Description / Why / Verify / Implement**.

> *You cannot tune what you cannot measure. The first three requirements exist to make the **measure-first** method possible; skipping them turns tuning back into guessing.*

1. Metrics, monitoring & a repeatable baseline

• **Description:** The ability to capture, at will and repeatably, GC behavior (GC logs), WebLogic

runtime metrics (thread pool + JDBC data-source MBeans), OUAF batch elapsed/throughput data, and an Oracle Database performance snapshot (**AWR/ASH**, which requires the **Diagnostics Pack** license).

• **Why required:** The method baselines the stack, isolates the saturated layer from evidence, and validates each change against measured **gates**. Without a repeatable baseline there is no "before" to prove a "gain."

• **How to verify:** Confirm GC logging can be enabled on the JVMs; confirm WLST/JMX can read the server `ThreadPoolRuntime` and `JDBCDataSourceRuntimeMBean`; confirm you can generate an AWR report (`$ORACLE_HOME/rdbms/admin/awrrpt.sql`) or query V$ views; confirm batch run history is available (Batch Run Tree / `CI_BATCH_RUN` tables or the JMX batch client). Run `OC2MPERF-Discovery` — it collects the read-only baseline and tells you what it could and could not read.

• **How to implement if missing:** Enable GC logging (see the Tuning Method), enable JMX/WLST access, confirm the Diagnostics Pack entitlement for AWR/ASH (or fall back to Statspack / V$ views if not licensed — **verify your licensing**), and establish batch run-history access.

2. Environment & administrative access (WebLogic, OUAF, database)

• **Description:** Working access to the target environment: the **WebLogic Administration Console and WLST **(admin credentials), the** OUAF product OS user **shell on each node (to read `threadpoolworker.properties`, run `bedit.sh`, `submitbatchREST.sh`, `configureEnv.sh`, `initialSetup.sh`), and a** database account** that can read the performance views and (for remediation) alter the relevant objects.

• **Why required:** Every measurement and every tuning change is applied through these three interfaces; batch tuning in particular is done at the OS/OUAF layer, not the console.

• **How to verify:** Log in to the Administration Console; connect WLST to the Admin Server; `su` to the product user and source the environment (`splenviron.sh -e <your-environment>`); connect to the schema with SQL*Plus/SQLcl and select from `V$SESSION`.

• **How to implement if missing:** Have the infrastructure/DBA teams grant the accounts and roles, and install the client tooling (WLST is delivered with WebLogic; SQLcl/SQL*Plus for the database).

3. A representative, repeatable workload profile

• **Description:** A defined, **reproducible** workload that represents the pain — a load-test script for the online/web-service tiers, and/or a named set of **batch controls** (e.g. billing, meter-read processing, To-Do) run against a known data volume — plus the **success criteria** (target response time, batch window, throughput).

• **Why required:** Baseline and re-measure must use the **same** workload or the comparison is meaningless. A one-off "it felt slow" cannot be tuned against a gate.

• **How to verify:** Confirm you can replay the workload on the pilot environment on demand and that its inputs (data volume, concurrency, batch parameters) are pinned.

• **How to implement if missing:** Build the load-test scenario and/or select and script the representative batch job(s) with fixed parameters before baselining.

4. Certified, matched stack versions across the tuning path

• **Description:** OS, **JDK** (the JVM you are tuning), **WebLogic 14c**, **OUAF/C2M**, and **Oracle Database **at versions** certified together** for your C2M release. GC flags and default MBean values differ across JDK and WebLogic versions.

• **Why required:** Applying a GC flag or MBean setting valid for one JDK/WebLogic version to another can be a no-op or an error; supportability depends on a certified combination (see the global requirements).

• **How to verify:** `java -version` on the product user's path; WebLogic version from the console or `. setDomainEnv.sh; java weblogic.version`; C2M/OUAF version from the console **About**; database `SELECT * FROM v$version;`. Compare against the C2M certification matrix on MOS.

• **How to implement if missing:** Align to a certified combination **before** tuning — do not tune on an uncertified stack.

5. Change control, backups & a non-production pilot

• **Description:** A change window and approval for production; **verified backups** of the WebLogic domain config, OUAF product/config files (`threadpoolworker.properties`, data-source and JVM settings), and the database before any change; and a **non-production environment** to pilot on.

• **Why required:** Tuning changes are production-impacting. Every change in the method is **single, named, and reversible**, validated on the pilot first, then promoted under change control with a rollback ready.

• **How to verify:** Confirm a tested restore path for the domain config and OUAF config files, a DB backup/restore or flashback point, and a booked change window.

• **How to implement if missing:** Establish backups and change control before touching production; `OC2MPERF-ScopeHelper` helps you record the pilot scope and rollback targets up front.

6. Collaborating teams (DBA, infrastructure, app-dev)

• **Description:** Named contacts on the **DBA**, **infrastructure/OS**, and **application-development** teams, plus the environment's monitoring stack (e.g. Dynatrace), since a bottleneck can live in any layer and a fix in one layer (batch threads) has consequences in another (DB sessions).

• **Why required:** Cross-layer tuning needs cross-team coordination — e.g. raising batch threads requires the DBA to confirm `processes`/`sessions` headroom.

• **How to verify:** Confirm the contacts and escalation path.

• **How to implement if missing:** Identify the teams and the monitoring model before starting.

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

Tuning Method

This is the hands-on procedure. It is a **measure** → **isolate** → **tune** → **re-measure** → **validate** loop, run **once per bottleneck**, always on a **non-production pilot first**, with a **single, named, reversible** change per pass and a rollback ready for production. Names of tuning artifacts are given exactly; where a value or path is version-specific it is flagged **"verify for your version."**

> ***Golden rule.*** *Change **one layer at a time**, re-measure against the **same** workload, and keep or roll back based on a **measured gate** (see the Testing Guide). Never tune more than one knob per pass.*

Step 0 — Prepare (Assess)

1. Confirm the Requirements Guide is satisfied — metrics access, environment access, a repeatable

workload, matched versions, backups/change control.

1. Record scope and rollback targets with `OC2MPERF-ScopeHelper` (which nodes, which data sources,

which batch controls, current values to restore to).

1. **Back up** the domain config, the OUAF config files you may touch

(`threadpoolworker.properties`, JDBC/data-source and JVM settings), and the database.

Step 1 — Measure & baseline

Capture a repeatable baseline of the **whole stack** under the representative workload. Run `OC2MPERF-Discovery` (read-only) to collect it, and enable GC logging so the JVM layer is visible.

- **JVM / GC.** Enable GC logging on each WebLogic node and each OUAF thread pool worker JVM. On a

modern JDK (JDK 11/17 — **verify for your version**) use unified logging: `-Xlog:gc*:file=/path/gc-%p.log:time,uptime,level,tags`. On JDK 8 (**verify**): `-verbose:gc -Xloggc:/path/gc.log -XX:+PrintGCDetails -XX:+PrintGCDateStamps`. Add these to the node's Java options (`setUserOverrides.sh` / the managed server's **Server Start** → **Arguments**, or the OUAF worker's `-J` options). Record heap size, collector, and steady-state GC pause and GC-time fraction.

- **WebLogic pools.** Read the runtime MBeans (via WLST/JMX; `OC2MPERF-Discovery` does this read-only):

- **Thread pool** (`ServerRuntime` → `ThreadPoolRuntime`): `ExecuteThreadTotalCount`,

`HoggingThreadCount`, `StuckThreadCount`, `PendingUserRequestCount`, `Throughput`, `SharedCapacityForWorkManagers`.

- **JDBC data source** (`ServerRuntime` → `JDBCServiceRuntime` → `JDBCDataSourceRuntimeMBeans`):

`ActiveConnectionsCurrentCount`, `ActiveConnectionsHighCount`, `WaitingForConnectionCurrentCount`, `WaitingForConnectionHighCount`, `ConnectionDelayTime`, `CurrCapacity`, `NumUnavailable`.

- **Batch.** Capture elapsed time and throughput for the representative batch control(s) from the

Batch Run Tree / batch run tables, or the JMX batch client (`jmxbatchclient.sh -s` for a summary), along with the current thread pool worker layout and per-batch thread count.

- **Database.** Take an **AWR** snapshot spanning the workload (`awrrpt.sql`) — or query V$ views /

Statspack if the Diagnostics Pack is not licensed (**verify licensing**). Record the top wait events, DB CPU, and top SQL by elapsed time.

Write the numbers down. This baseline is the gate every later change is judged against.

Step 2 — Isolate the bottleneck

Read the baseline and identify the **one** saturated layer. Typical signatures:

Signature in the baseline	Constrained layer	Go to
GC time a high fraction of wall-clock; long/ frequent pauses; heap near `-Xmx` after full GC	**JVM / GC**	Step 3a
`PendingUserRequestCount` persistently > 0; `HoggingThreadCount` / `StuckThreadCount` rising; throughput flat while requests queue	**WebLogic threads / Work Managers**	Step 3b
`WaitingForConnectionCurrentCount` > 0; `ActiveConnectionsHighCount` at `Maximum Capacity`; `ConnectionDelayTime` high	**JDBC connection pool**	Step 3c
Batch elapsed dominated by one job; threads idle waiting; window overruns	**Batch throughput**	Step 3d
AWR top waits are I/O / row-lock / library-cache; a few SQL statements dominate elapsed	**SQL / Database**	Step 3e

If two layers look saturated, tune the **lower** one first (a fix there often relieves the layer above) and re-measure before touching the other.

Step 3 — Tune the constraint (one layer, one change)

3a — JVM / GC

1. **Right-size the heap.** Set `-Xms` equal to `-Xmx` to avoid runtime heap resizing pauses; size

`-Xmx` from the baseline (post-full-GC live-set with headroom). Confirm the node has RAM headroom so the heap is not swapped.

1. **Choose/tune the collector.** On JDK 11/17, **G1GC** is the default (**verify for your version**);

set a pause target with `-XX:MaxGCPauseMillis` rather than hand-tuning generations. Avoid stacking contradictory flags. Size **metaspace** (`-XX:MetaspaceSize` / `-XX:MaxMetaspaceSize`) from the baseline to stop metaspace-driven full GCs.

1. Keep GC logging on so Step 4 can compare pause distribution and GC-time fraction.

Apply the same discipline to **OUAF thread pool worker** JVMs (their heap/GC is set in the worker's Java options), not just WebLogic managed servers.

> *Reference: Oracle WebLogic Server 14.1.1 **Tuning Performance** guide; JDK GC tuning guide for your JDK version. **Verify default collector and flag syntax for your JDK.***

3b — WebLogic threads / Work Managers

WebLogic 9.0+ uses a **self-tuning** thread pool — you do **not** set a fixed thread count. Instead, shape it with **Work Managers** and constraints:

1. Confirm the queue is genuinely thread-starved (pending requests with CPU/heap headroom), not blocked

downstream (a full connection pool or slow SQL looks identical — rule those out first).

1. Define a **Work Manager** for the C2M application (or module) and, where justified, a

`min-threads-constraint` (guarantee threads for a critical path) and/or `max-threads-constraint` (cap a component so it cannot starve others). Set these via the Administration Console or WLST.

1. Investigate **stuck/hogging threads** at the source — a rising `StuckThreadCount` usually means a

downstream wait (DB/connection pool), not a need for more threads. Tune `StuckThreadMaxTime` only to change *detection*, not to mask a real stall.

> *Reference: WebLogic 14.1.1 "Using Work Managers to Optimize Scheduled Work" and the Tuning Performance guide. The self-tuning pool's max is governed by `SelfTuningThreadPoolSizeMax` (default **400**; **verify for your version**).*

3c — JDBC connection pool

Tune each C2M data source from its runtime evidence:

1. **Capacity.** Raise `Maximum Capacity` to clear `WaitingForConnectionCurrentCount`, and set

`Initial Capacity` equal to `Maximum Capacity` so connections are pre-created (avoids first-use latency). **But** cap it against the database's `sessions`/`processes` headroom — confirm with the DBA before raising. An over-large pool moves the bottleneck to the database.

1. **Validation.** Enable **Test Connections on Reserve** with a **Test Frequency** and a short trusted

window so healthy connections skip the test; this trades a little overhead for reliability under partial DB outages.

1. **Statement cache.** Set a **Statement Cache Size** (type `LRU`) sized to the application's distinct

prepared statements to cut parse/CPU on the database.

1. **Shrinking / timeouts.** Tune **Shrink Frequency**, **Connection Reserve Timeout**, and

Inactive Connection Timeout so the pool does not thrash open/close under bursty load.

> *Reference: WebLogic 14.1.1 "Tuning Data Source Connection Pools." **Verify attribute defaults for your version.***

3d — Batch throughput (OUAF thread pool workers)

Batch is tuned at the OUAF/OS layer, not the WebLogic console:

1. **Thread pool worker layout.** Size the named thread pools with

`threadpoolworker.sh -p DEFAULT=<n>,<POOL>=<n>` (or configure via `bedit.sh`), matching pool sizes to CPU cores available to the worker JVMs. Distribute workers across nodes for volume batches (CLUSTERED mode); use THIN mode only for isolated single-JVM runs.

1. **Per-batch threads.** Set the thread count per batch control with `submitbatchREST.sh -c <threads>`

(the modern submitter; `submitjob.sh -c` is legacy). More threads only help if the batch partitions cleanly and the DB/connection pool can absorb the extra concurrency.

1. **Commit interval / chunking.** Tune the batch's commit (`chunk`) interval — too small floods the DB

with commits, too large lengthens locks and undo. Set per the batch control's guidance.

1. **Balance against layers 2 and 4.** Every added batch thread consumes a JDBC connection and a DB

session. Confirm the connection pool (Step 3c) and DB `processes`/`sessions` (Step 3e) can carry the new concurrency **before** raising thread counts.

> *Reference: OUAF Server Administration Guide — Batch Server Configuration, `threadpoolworker.sh`, `submitbatchREST.sh`, `bedit.sh`. Optimal thread counts are workload- and data-specific — **verify for your batches**. Some batch-tuning notes are **MOS-gated**; they are pointed to, not reproduced.*

3e — SQL / Database

1. From the AWR/ASH top-SQL and top-wait findings, take the **highest-elapsed** statement(s) first.

2. Get the **execution plan** (`DBMS_XPLAN.DISPLAY_CURSOR`), check for stale/missing **statistics**

(`DBMS_STATS`) and missing/unusable **indexes**, and confirm the plan is not regressed.

1. Apply the **narrowest** fix — refresh stats, add/repair an index, or fix a plan — never a schema

change to product-owned objects without vendor guidance (custom/CM objects only).

1. Confirm **database capacity** headroom used by the pools and batch: `processes`, `sessions`, undo,

temp, and CPU. Raising batch threads or pool capacity is safe only within this headroom.

> *Product-schema changes to C2M objects are vendor-governed — **confirm with support/MOS** before altering product tables or indexes. Do not invent index changes on product objects.*

Step 4 — Re-measure & validate

Re-run the **same** workload with `OC2MPERF-Validation` (read-only) and compare against the Step 1 baseline:

• Did the **target metric** cross its **gate** (Testing Guide)? e.g. GC-time fraction down, `WaitingForConnectionCurrentCount` at zero, batch elapsed reduced, top-SQL elapsed reduced.

• Did any **other** layer regress? (More batch threads must not create connection waits or DB saturation; a bigger heap must not increase pause time past target.)

Keep the change only if it passes the gate with no regression; otherwise **roll back** to the recorded value and try a different hypothesis.

Step 5 — Expand (promote & repeat)

1. Promote the validated change **Dev** → **Test** → **Prod** under change control, re-measuring at each stop.
2. Update the recorded baseline to the new steady state.
3. Return to **Step 2** for the next bottleneck. Stop when every layer meets its service target under the representative workload.

Generate an evidence pack for sign-off with `OC2MPERF-Report` (read-only) — baseline vs. current, per-layer, with the gate outcomes.

Testing Guide — Measured Gates

Tuning is only "done" when it is **proven**. Every change is judged against a **measured pass/fail gate** derived from the Step 1 baseline, re-measured on the **same** workload, on the **pilot first**. A change that does not pass its gate — or that passes but regresses another layer — is **rolled back**.

> ***Comparison rule.*** *Baseline and re-measure must use the identical workload (same data volume, concurrency, batch parameters). Use* `OC2MPERF-Discovery` *for the baseline and* `OC2MPERF-Validation` *for the re-measure so the two captures are comparable, then* `OC2MPERF-Report` *for the evidence pack.*

Gate targets are relative, not absolute

There is no universal "good" heap size or thread count. The gate is a **relative improvement against your own baseline** with **no regression elsewhere**. Set the target percentages with your stakeholders from the workload's success criteria; the tables below give the metric and the direction.

Layer gates

JVM / GC

Metric (from GC logs)	Pass condition
GC time as fraction of wall-clock	**Decreased** vs. baseline and below your target (commonly a low single-digit %)
Steady-state pause time	Within `-XX:MaxGCPauseMillis` target; no sustained full-GC storms
Post-full-GC live heap	Comfortably below `-Xmx` (headroom remains)

WebLogic threads / Work Managers

Metric (`ThreadPoolRuntime`)	Pass condition
`PendingUserRequestCount`	At or near **zero** under the workload
`StuckThreadCount` / `HoggingThreadCount`	Not rising; no new stuck threads introduced
`Throughput`	**Increased** or unchanged (never regressed)

JDBC connection pool

Metric (`JDBCDataSourceRuntimeMBean`)	Pass condition
`WaitingForConnectionCurrentCount`	**Zero** under the workload
`ActiveConnectionsHighCount` vs. `Maximum Capacity`	High-water below max (headroom remains)
`ConnectionDelayTime` / reserve latency	**Decreased** vs. baseline
`NumUnavailable`	Zero

Batch throughput

Metric	Pass condition
Batch elapsed time (representative control)	**Decreased** vs. baseline and within the batch window
Throughput (records/units per minute)	**Increased** vs. baseline
Thread utilization	Threads doing work, not idle-waiting on connections/locks

SQL / Database

Metric (AWR/ASH)	Pass condition
Top-SQL elapsed time (tuned statement)	**Decreased** vs. baseline
Top wait events	Target wait reduced; no new dominant wait introduced
DB CPU / sessions / processes	Within capacity headroom; no session/process exhaustion

No-regression gate (mandatory, every pass)

A per-layer pass is invalid if it pushed load into another layer. After each change, confirm **none** of the following regressed past its baseline:

- GC pause/time fraction (Step 3d batch threads must not induce heap pressure).
- Connection-pool waits (Step 3d/3b must not create `WaitingForConnectionCurrentCount`).
- DB sessions/processes/CPU (Step 3c/3d must not exhaust DB capacity).
- WebLogic stuck/hogging threads (no new stuck threads from any change).

Production go/no-go gate

Before promoting to production:

1. The change **passed its layer gate** on the pilot with the representative workload.
2. The **no-regression gate** passed.
3. A **rollback** to the recorded prior value is verified and a change window is booked.
4. `OC2MPERF-Report` shows baseline-vs-current evidence for the change.

If any item fails, it is **no-go** — roll back and re-diagnose.

Test execution checklist

- Baseline captured with `OC2MPERF-Discovery` on the pilot, numbers recorded.
- Bottleneck isolated to a single layer.
- One named, reversible change applied to that layer only.
- Re-measure with `OC2MPERF-Validation` on the **same** workload.
- Layer gate **and** no-regression gate both pass.
- Evidence pack generated with `OC2MPERF-Report`.
- Promote under change control, or roll back and re-diagnose.

Troubleshooting Guide

Common tuning traps across the four layers, as **symptom** → **likely cause** → **what to do**. The recurring theme: a symptom in one layer usually has its cause in another — measure before you tune.

> *Run `OC2MPERF-Discovery` (read-only) whenever a symptom appears — nearly every entry below is diagnosed from the metrics it collects.*

Cross-cutting

Symptom: "C2M is slow" but you don't know where.

• **Cause:** No baseline; symptom pattern-matched instead of diagnosed.

• **Fix:** Capture the baseline (Step 1), then use the isolation table (Step 2) to find the one saturated layer. Do not change anything until a layer is isolated.

Symptom: A tuning change "helped" but overall throughput didn't improve.

• **Cause:** The bottleneck moved to the next layer (e.g. more batch threads now saturate the DB).

• **Fix:** Re-run `OC2MPERF-Validation` and check the **no-regression gate**. Tune the newly constrained layer; keep the earlier change only if it passed its own gate.

JVM / GC

Symptom: Periodic freezes / high response-time spikes; long GC pauses in the GC log.

• **Cause:** Full-GC storms — undersized heap, metaspace pressure, or a mismatched collector.

• **Fix:** Set `-Xms = -Xmx`, size the heap from post-full-GC live set with headroom, size metaspace, and set a G1 pause target (`-XX:MaxGCPauseMillis`). Confirm RAM headroom so the heap is not swapped. **Verify GC flags for your JDK version.**

Symptom: Heap grows to `-Xmx` and stays there; frequent full GCs.

• **Cause:** Live set genuinely near the ceiling, or a leak.

• **Fix:** Re-baseline live set after a full GC; raise `-Xmx` only within RAM headroom. If it keeps climbing, capture a heap dump and involve app-dev — this is a leak, not a tuning problem.

WebLogic threads / Work Managers

Symptom: `PendingUserRequestCount` **> 0, requests queue, but CPU and heap are idle.**

• **Cause:** Threads blocked **downstream** — usually a full connection pool or slow SQL — not a lack of threads.

• **Fix:** Check the JDBC pool (`WaitingForConnectionCurrentCount`) and top SQL first. Adding threads here only deepens the downstream queue.

Symptom: `StuckThreadCount` **/** `HoggingThreadCount` **rising.**

• **Cause:** Threads waiting on a long downstream operation (DB, connection reserve, external call).

• **Fix:** Find the wait's source and fix *that*. Adjust `StuckThreadMaxTime` only to tune detection, never to mask a real stall.

JDBC connection pool

Symptom: `WaitingForConnectionCurrentCount` **> 0;** `ActiveConnectionsHighCount` **at** `Maximum Capacity`**.**

• **Cause:** Pool too small for the concurrency (often after raising batch threads).

- **Fix:** Raise `Maximum Capacity` (and `Initial Capacity` to match) **within DB session headroom** —

confirm `processes`/`sessions` with the DBA first. If the DB can't take more sessions, reduce upstream concurrency instead.

Symptom: First requests after idle are slow; connection errors after a brief DB blip.

- **Cause:** Connections created on demand; stale connections handed out.
- **Fix:** Set `Initial Capacity = Maximum Capacity`; enable **Test Connections on Reserve** with a

test frequency and a short trusted window.

Symptom: High DB parse/CPU with repetitive statements.

- **Cause:** No statement cache.
- **Fix:** Set an `LRU` **Statement Cache Size** sized to the app's distinct prepared statements.

Batch throughput

Symptom: A batch overruns its window; threads sit idle-waiting.

- **Cause:** Threads starved of connections, blocked on row locks, or the job doesn't partition.
- **Fix:** Confirm the connection pool and DB headroom (layers 2 & 4) before adding threads. Only raise

`submitbatchREST.sh -c` if the batch partitions cleanly and downstream capacity exists.

Symptom: Raising thread count made batch *slower* or spiked DB waits.

- **Cause:** Concurrency exceeded DB/connection capacity → contention, locking, or session exhaustion.
- **Fix:** Back the thread count down, size the connection pool and DB sessions for the intended

concurrency, and re-test. Tune the commit/`chunk` interval — too small floods commits, too large lengthens locks/undo.

SQL / Database

Symptom: AWR shows a few statements dominating elapsed time.

- **Cause:** Bad plan, stale/missing stats, or a missing index.
- **Fix:** `DBMS_XPLAN.DISPLAY_CURSOR` the statement, refresh stats (`DBMS_STATS`), add/repair an

index — **only on custom/CM objects**; product-schema changes are vendor-governed (confirm via support/MOS).

Symptom: Top waits are session/process related; new sessions fail.

- **Cause:** `processes`/`sessions` exhausted — often from oversized pools or too many batch threads.
- **Fix:** Right-size pools and batch concurrency to the DB's session headroom, or raise

`processes`/`sessions` with the DBA (requires a DB restart).

Escalation

• **MOS-gated notes/patches:** where a fix references a My Oracle Support note or patch, open a service

request — this solution points to gated content, it does not reproduce it.

• **Product-schema or product-code issues:** engage Oracle support; do not alter product-owned objects.

• **Suspected memory leaks / defects:** capture heap/thread dumps and the GC log and involve

application-development and support.

Best Practices

Repeatable, evidence-based tuning for C2M on WebLogic 14c. These are the habits that keep tuning safe, honest, and durable.

Measure first, always

• **No baseline, no change.** Never adjust a knob without a recorded "before." The baseline is the

only thing that lets you prove a gain — or catch a regression.

• **Same workload, every time.** Baseline and re-measure with the identical workload (data volume,

concurrency, batch parameters). A different workload invalidates the comparison.

• **Diagnose the layer, don't pattern-match the symptom.** Slow SQL, a starved thread pool, and an

exhausted connection pool all *look* like "C2M is slow." Read the evidence and isolate the real constraint before touching anything.

One change at a time

• **Single, named, reversible.** Each tuning pass changes exactly one layer, records the prior value,

and has a rollback ready. Bundled changes make it impossible to attribute the result.

• **Tune the lower layer first.** When two layers look saturated, fix the lower one (DB/connection

pool) and re-measure — it often relieves the layer above and saves you a change.

Respect layer coupling

• **Batch threads cost connections and DB sessions.** Before raising `submitbatchREST.sh -c` or a

thread pool size, confirm the JDBC pool `Maximum Capacity` and the database `processes`/`sessions` can carry the extra concurrency. Coordinate with the DBA.

• **A bigger pool can overload the database.** Sizing `Initial Capacity = Maximum Capacity` removes

first-use latency but must stay within DB session headroom.

• **More heap is not free.** Larger heaps can lengthen GC pauses; keep the pause target and confirm

RAM headroom so the heap is never swapped.

Prefer self-tuning and shaping over fixed counts

- WebLogic 9.0+ uses a **self-tuning** thread pool — shape it with **Work Managers** and constraints

rather than forcing fixed thread counts. Investigate stuck/hogging threads at the source (usually a downstream wait), don't just add threads.

Production discipline

- **Pilot first.** Validate every change on a non-production environment against a measured gate before

it reaches production, then promote under change control.

- **Back up before you touch.** Domain config, OUAF config files (`threadpoolworker.properties`, JVM

and data-source settings), and the database — verified restore path.

- **Dry-run remediation.** Run `OC2MPERF-Remediation` in its default **dry-run** mode first, review

the planned change, then apply deliberately in a change window.

Version and licensing hygiene

- **Verify for your version.** GC flags, MBean paths, OUAF utility options, and default values shift

across JDK / WebLogic / OUAF releases. Confirm against the certified matrix and the docs for your exact versions — never carry a value across versions blindly.

- **Respect gated content and licensing.** AWR/ASH need the Diagnostics Pack; some tuning notes/patches

are MOS-gated. Point to them; don't assume entitlement.

- **Don't alter product-owned schema.** Index/stat/plan fixes on C2M product objects are

vendor-governed — confirm with support/MOS; free-hand only custom/CM objects.

Keep the evidence

- **Record every pass.** Baseline, change, re-measure, gate outcome, keep/rollback decision — build the

evidence pack with `OC2MPERF-Report` so tuning is auditable and repeatable next quarter.

- **Re-baseline after promotion.** Once a change is live, the new steady state becomes the baseline for

the next bottleneck.

Deliverables Summary

What you receive in this solution and how the pieces fit the **measure-first** method.

Documentation

Document	What it gives you
Solution Overview	The C2M performance model (JVM/GC → WebLogic pools → batch → database) and the measure-first tuning method.
Requirements Guide	What you need before tuning — metrics/monitoring access, environment & admin access, a repeatable workload, matched versions, backups/change control — each with how to *verify*.
Tuning Method	The ordered hands-on procedure: baseline → isolate the bottleneck → tune JVM/GC, WebLogic thread & connection pools, batch thread pools, SQL/DB → re-measure & validate → expand.
Testing Guide	The measured pass/fail **gates** per layer, the mandatory no-regression gate, and the production go/no-go gate.
Troubleshooting Guide	Symptom-to-cause-to-fix for the common tuning traps across all four layers.
Best Practices	The habits that keep tuning safe, honest, and repeatable.

Automation scripts (WLST/Jython, shell, SQL)

Five scripts authentic to C2M and WebLogic administration. The three that touch live systems are **read-only**; remediation is **dry-run by default**.

Script	Type	Mode	Purpose
OC2MPERF-Discovery	WLST + SQL	Read-only	Collect the baseline: JVM/GC settings, WebLogic thread-pool and JDBC data-source runtime metrics, batch layout, and DB performance snapshot.
OC2MPERF-Validation	WLST + SQL	Read-only	Re-measure the same metrics after a change so it can be compared to the baseline, per node/data source.
OC2MPERF-Report	shell	Read-only	Build the health/coverage evidence pack — baseline vs. current, per layer, with gate outcomes.

Script	Type	Mode	Purpose
OC2MPERF-Remediation	WLST/shell	**Dry-run default**	Apply a single validated tuning change (heap/GC arg, data-source capacity, thread count) — prints the planned change unless explicitly told to apply.
OC2MPERF-ScopeHelper	shell	Read-only helper	Record the pilot scope and rollback targets: which nodes, data sources, and batch controls are in play and their current values.

Each script carries a native-comment header (**PURPOSE / HOW TO RUN / EXPECTED RESULTS / NOTES**) and uses placeholders without angle brackets (e.g. `your-domain`). Scripts are delivered as WLST (Jython) `.py`, shell `.sh`, and SQL `.sql` — **not** PowerShell.

How they map to the method

- **Assess** → `OC2MPERF-ScopeHelper` records scope and rollback targets.
- **Measure & baseline** (Step 1) → `OC2MPERF-Discovery`.
- **Tune** (Step 3) → apply by hand or with `OC2MPERF-Remediation` (dry-run first).
- **Re-measure & validate** (Step 4) → `OC2MPERF-Validation`, then gates in the Testing Guide.
- **Sign-off / expand** (Step 5) → `OC2MPERF-Report` evidence pack.

Outcome

A slow C2M environment turned into a **measured, optimized** one — JVM, pools, batch, and SQL tuned to the real bottleneck, every change proven against a baseline on a pilot before it reaches production.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MPERF-Discovery.py

Purpose: Collect a repeatable performance baseline via WLST/JMX WITHOUT changing anything. Reads JVM/GC settings, the WebLogic self-tuning thread pool runtime, and every JDBC data-source connection-pool runtime for each managed server in the domain. This is Step 1 (Measure & baseline) of the measure-first method; the numbers it prints are the gate that later changes are validated against. Fill the placeholders (admin-url, admin-user) for your-domain before running.

How to run: java weblogic.WLST OC2MPERF-Discovery.py (prompts for the admin password; connects read-only to the Admin Server, then loops the managed servers. Pair with the DB baseline in OC2MPERF-Discovery.sql.)

OC2MPERF-Remediation.py

Purpose: Apply a single, validated tuning change to a WebLogic 14c JDBC data-source connection pool from the measure-first method: Maximum Capacity, Initial Capacity, Test-Connections-On-Reserve, and Statement Cache Size. It changes ONE data source per run and, by default, only PRINTS the planned edit (dry-run). It applies nothing unless explicitly told to. Use it after a change has passed its gate on the pilot. Fill placeholders (admin-url, admin-user, datasource-name) for your-domain.

How to run: Dry-run (default, changes nothing): java weblogic.WLST OC2MPERF-Remediation.py Apply for real (edits + activates the change): java weblogic.WLST OC2MPERF-Remediation.py apply (prompts for the admin password. Back up the domain config first; run in a change window.)

OC2MPERF-Report.sh

Purpose: Assemble a single evidence pack for tuning sign-off from read-only sources: the WLST baseline/re-measure output (OC2MPERF-Discovery / OC2MPERF-Validation), the OUAF batch layout, the JVM/GC log summary, and the database performance snapshot (OC2MPERF-Report.sql). Nothing is changed. Use it to show baseline-vs-current per layer with gate outcomes for the change-control record. Source the environment first (splenviron.sh -e your-environment).

How to run: . splenviron.sh -e your-environment ./OC2MPERF-Report.sh -b baseline-discovery.txt -c current-validation.txt -o report-out-dir (-b baseline capture, -c current capture, -o output directory; -g points at a GC log to summarize, -d runs the DB snapshot via OC2MPERF-Report.sql. All inputs are read-only.)

OC2MPERF-Report.sql

Purpose: Companion to OC2MPERF-Report.sh: capture the database side of the baseline/evidence pack WITHOUT changing anything. Reports the top SQL by elapsed time, the top wait events, and the session/process capacity headroom that pool and batch tuning depend on. Use it to see whether the database is the bottleneck (layer 4) and whether it has headroom for more connections/threads. Connect as a monitoring account for the C2M schema/instance.

How to run: sqlplus /nolog @OC2MPERF-Report.sql (it issues CONNECT and prompts for credentials; or run it from OC2MPERF-Report.sh -d. Requires read access to V$ views; AWR views additionally require the Diagnostics Pack.)

OC2MPERF-ScopeHelper.sh

Purpose: Before a tuning pass, capture WHAT is in scope and HOW to roll it back, so every change is single, named, and reversible (Step 0 of the method). It records the target nodes, JDBC data sources, and batch controls, snapshots the current OUAF batch layout and JVM/data-source settings it can read, and writes a scope manifest you attach to the change-control record. Source the environment first (splenviron.sh -e your-environment). Read-only.

How to run: . splenviron.sh -e your-environment ./OC2MPERF-ScopeHelper.sh -n "node1,node2" -s "your-datasource-name" -c "BATCHCTRL1,BATCHCTRL2" -o scope-out-dir (-n nodes, -s data sources, -c batch controls in scope; -o output dir. All inputs recorded; current values it can read are snapshotted as the rollback baseline.)

OC2MPERF-Validation.py

Purpose: Re-capture the same WebLogic thread-pool and JDBC connection-pool runtime metrics that OC2MPERF-Discovery captured, so a tuning change can be compared against the baseline and judged against the measured gates. This is Step 4 (Re-measure & validate) of the method. It is deliberately the SAME reads as Discovery, run on the SAME workload, so the two captures are directly comparable. Fill the placeholders (admin-url, admin-user) for your-domain, and re-run under the identical workload used for the baseline.

How to run: java weblogic.WLST OC2MPERF-Validation.py (prompts for the admin password; connects read-only. Diff the output against the OC2MPERF-Discovery baseline, then apply the Testing Guide gates.)